Java Interface

9/15/2020

- SOBUZ
- 2
- Interface
- Interface is used to specify what a class must do but not how to do.
- Interface is used for making a full abstract class.
- Interface is the blueprint of the class.
- Similar to class but does not have instance variables.
- Interfaces’ methods are declared without any body.

The only fields that can appear in an interface must be declared both static and final. By default these two keywords are added even when not specified.

- Keyword “interface” is used.

9/15/2020

- SOBUZ
- 3
- Interface
- By default all methods are public and abstract in interface.
- implement keyword is used for the implementation of interface.
- We can’t create the object of the interface.
- Multiple inheritance can be implemented by interface.
- Benefits:
- Used to achieve “one interface multiple methods” aspect of polymorphism.
- Used to achieve loose coupling.

Syntax for interface

- access interface name {
- return-type method-name1(parameter-list);
- return-type method-name2(parameter-list);
- type final-varname1=value;
- type final-varname2=value;

```
> // ...
```

- return-type method-nameN(parameter-list);
- Type final-varnameN=value;
- }

When a class implements interface, consider the class as if it is signing a contract, agreeing to perform the specific behaviours of the interface. If the class does not perform all the behaviours of the interface, the class must declare itself as abstract.

- Example:
- interface Animal {
- public void eat();
- public void travel();
- }

```
> public class MammalInt implements Animal {
```

- public void eat() {

```
> System.out.println(“Mammal eats”);
```

- }
- public void travel() {

```
> System.out.println(“Mammal travels”);
```

- }
- public void noOfLegs() {

```
> return 0;
```

- }

public static void main(String args[])

- {
- MammalInt m = new MammalInt();
- m.eat();
- m.travel();
- }
- }
- Output:
- Mammal eats
- Mammal travels

When overriding methods defined in interfaces, there are several rules to be followed:

Checked exceptions should not be declared on implementation methods other than the ones declared by the interface method or subclasses of those declared by the interface method.

- The signature of the interface method and the return type should be maintained when overriding the methods.
- An implementation class itself can be abstract and if so, interface methods need not be implemented..

Rules for implementing interface

- A class can implement more than one interface at a time.
- A class can extend only one class, but implement many interfaces.
- An interface can extend another interface, in a similar way as a class can extend another class.

Extending interfaces

- An interface can extend another interface in the same way that a class can extend another class.

The extends keyword is used to extend an interface, and the child interface inherits the methods of the parent interface.

Example:

- public interface Sports {
- public void setHomeTeam(String name);
- public void setVisitingTeam(String name);
- }

```
> public interface Football extends Sports {
```

- public void homeTeamScored(int points);
- public void visitingTeamScored(int points);
- public void endOfQuarter(int quarter);
- }

```
> public interface Hockey extends Sports {
```

- public void homeGoalScored();
- public void visitingGoalScored();
- public void endOfPeriod(int period);
- public void ovretimePeriod(int ot);
- }

The Hockey interface has four methods. It inherits two from sports. Thus a class that implements Hockey needs to implement all six methods. Similarly a class that implements Football needs to define five methods.

Extending multiple interfaces

- A Java class can only extend one parent class.

Multiple inheritence is not allowed. Inerfaces are not classes, however, an interface can extend more than one parent interface.

- The extend keyword is used once, and the parent interfaces are declared in a comma seperated list.
- For example ,if the Hockey interface extended both Sports and Event, it would be declared as –
- public interface Hockey extends Sports, Event

Difference between abstract class and interface

- Abstract class
- An abstract class can have a method body(non-abstract methods).
- An abstract class can have instance variables.
- An abstract class can have the constructor.
- An abstract class can have static methods.
- We can extend one abstract class.
- The abstract class can provide the implementation of the interface.
- The abstract keyword is used to declare an abstract class.
- An abstract class can be extended using keyword extends.
- A java abstract class can have class members like private, protected, etc.
- Example:
- public abstract class Shape {
- public abstract void draw();
- }

Interface:

- The interface has only abstract methods.
- An interface cannot have instance variables.
- The interface cannot have the constructor.
- The interface cannot have static methods.
- We can implement multiple interfaces.
- The interface can’t provide the implementation of the abstract class.
- The interface keyword is used to declare an interface
- An interface can extend another Java interface only.
- An interface class can be implemented using keyword implements.
- Members of a Java interface are public by default.
- Example:

```
> public interface Drawable {
```

- void draw();
- }